

# FathomNet 2026 — Master Plan v2

**Today:** Friday, April 17, 2026 **Kaggle deadline:** May 7, 2026 (20 days) **Budget:** up to \$300 (target ~\$230, leave ~\$70 as headroom)  
**Stance:** Best possible model AND learning experience — no shortcuts

**Changelog from v1:** Folded in Equalized Focal Loss, Repeat Factor Sampling, class-aware copy-paste, Soft Teacher (replacing plain pseudo-labeling), multi-scale training, underwater preprocessing, SAHI tiled inference, stratified 5-fold CV. SSL pretraining is now fully included (not skipped), and full-training phase uses 5-fold ensemble.

## Do This First (April 17-18)

1. **Register for CLEF by April 23** — [clef-labs-registration.dipintra.it/registrationForm.php](https://clef-labs-registration.dipintra.it/registrationForm.php)
2. **Join the Kaggle competition** -> [kaggle.com/competitions/fathomnet-2026](https://kaggle.com/competitions/fathomnet-2026)
3. **Accounts:** RunPod (add \$100 credit to start), W&B (free), Kaggle API token downloaded
4. **Tooling:** Cursor installed with Remote-SSH extension, SSH key generated and added to RunPod
5. **Start Phase 0 EDA** — no GPU needed

Miss CLEF registration = no official ranking regardless of Kaggle score.

## Budget Allocation — up to \$300 (target ~\$230)

| Phase                                                  | Where            | GPU       | Hours             | Cost          |
|--------------------------------------------------------|------------------|-----------|-------------------|---------------|
| 0 — EDA + setup                                        | Local (Cursor)   | None      | 0                 | \$0           |
| 1 — SSL pretraining (SimCLR + DINOv2 fine-tune)        | RunPod A100 80GB | A100 80GB | ~18               | ~\$45         |
| 2 — Baselines                                          | Kaggle free      | T4        | ~4                | \$0           |
| 3 — PU learning (EFL + RFS + Soft Teacher)             | RunPod RTX 4090  | RTX 4090  | ~15               | ~\$12         |
| 4 — Architecture + aug (class copy-paste, multi-scale) | RunPod RTX 4090  | RTX 4090  | ~24               | ~\$18         |
| 5 — Hierarchical + pseudo R2                           | Mix (T4 + 4090)  | T4 + 4090 | ~8                | ~\$5          |
| 6 — 5-fold YOLOv11m + RT-DETR                          | RunPod A100 40GB | A100 40GB | ~40               | ~\$80         |
| 7 — SAHI + TTA + ensemble + calibration                | RunPod RTX 4090  | RTX 4090  | ~15               | ~\$11         |
| Buffer (reruns, debugging, opportunistic extras)       | —                | —         | —                 | ~\$60         |
| <b>Target total</b>                                    |                  |           | <b>~124 hours</b> | <b>~\$231</b> |
| <b>Headroom available</b>                              |                  |           |                   | <b>~\$70</b>  |

**Max is \$300, but don't spend it for the sake of spending it.** The ~\$70 of headroom beyond the \$231 target is there for: 1. A Phase 6 fold that crashes and needs rerunning (most likely use) 2. Unexpected debugging time (competitions always have surprises) 3. Pulling a high-value item from the backup plan mid-competition (temperature scaling, per-class thresholds — almost free) 4. Extra Soft Teacher iterations if the first attempt underperforms

If nothing goes wrong, total spend lands around \$230 and you have \$70 unspent. That's fine.

## Timeline — 20 Days

| Days | Dates     | Phase | Focus                              |
|------|-----------|-------|------------------------------------|
| 1-2  | Apr 17-18 | 0     | EDA + CLEF reg + environment setup |
| 3-5  | Apr 19-21 | 1     | SSL pretraining                    |

|       |           |                 |                               |
|-------|-----------|-----------------|-------------------------------|
| 6-7   | Apr 22-23 | 2               | Baselines + first submission  |
| 8-10  | Apr 24-26 | 3               | EFL + RFS + Soft Teacher      |
| 11-13 | Apr 27-29 | 4               | Architecture + aug ablations  |
| 14    | Apr 30    | 5               | Hierarchical + pseudo R2      |
| 15-18 | May 1-4   | 6               | Full 5-fold training          |
| 19    | May 5     | 7               | SAHI + ensemble + calibration |
| 20    | May 6     | Buffer          | Final submission, last fixes  |
| 21    | May 7     | <b>DEADLINE</b> | Submit by 11:59 PM UTC        |

**Golden rule:** One variable changed per experiment. Otherwise attribution is impossible.

---

## The Core Challenge (Read First)

---

Standard YOLO assumes: labeled box = object present, no box = background.

FathomNet labels are **incomplete**: a region with no box might be background OR it might be an unlabeled organism. Training naively penalizes correct detections in unlabeled regions.

On top of that, the 32 categories are **long-tailed** — some classes have thousands of instances, others have tens. And the 2025 post-mortem revealed a **distribution shift**: test objects are on average smaller than training objects.

So three problems to solve simultaneously: 1. **Positive-unlabeled (PU)** — incomplete labels 2. **Long-tail class imbalance** — rare classes under-represented 3. **Small-object / scale shift** — test objects smaller than training

Every differentiator in this plan addresses at least one.

---

## Phase 0 — EDA and Setup

---

**Days 1-2 • Local Cursor • No GPU • \$0**

### Goals

- Class frequency -> compute RFS repeat factors for Phase 3
- PU severity -> determines how critical PU losses are
- Object size distribution -> informs input resolution and SAHI tile size
- Stratified 5-fold split (used in Phase 6)
- Marine taxonomy tree (used in Phase 5)

### Key EDA script

```

import json
import numpy as np
from collections import Counter, defaultdict

with open("dataset_train.json") as f:
    data = json.load(f)

cat_names = {c['id']: c['name'] for c in data['categories']}
cat_counts = Counter(a['category_id'] for a in data['annotations'])

# 1. Class distribution - RFS needs this
print("Class counts (sorted):")
for cat_id, count in sorted(cat_counts.items(), key=lambda x: -x[1]):
    print(f"{cat_names[cat_id]:35s}: {count:5d}")

# 2. Compute RFS repeat factors per image (used in Phase 3)
# t = threshold frequency; classes rarer than t get upweighted
T = 0.001 # typical LVIS setting; tune from 0.001 to 0.01
total_instances = sum(cat_counts.values())
class_freq = {c: n/total_instances for c, n in cat_counts.items()}
# Repeat factor per class
class_rf = {c: max(1.0, np.sqrt(T / f)) for c, f in class_freq.items()}

anns_by_image = defaultdict(list)
for a in data['annotations']:
    anns_by_image[a['image_id']].append(a)

# Per-image repeat factor = max over classes present
image_rf = {}
for img_id, anns in anns_by_image.items():
    classes_in_image = set(a['category_id'] for a in anns)
    image_rf[img_id] = max(class_rf[c] for c in classes_in_image)

# Save for Phase 3
import pickle
with open('data/rfs_repeat_factors.pkl', 'wb') as f:
    pickle.dump(image_rf, f)

# 3. PU severity
annotated = set(a['image_id'] for a in data['annotations'])
single_cat = sum(1 for img_id, anns in anns_by_image.items()
                 if len(set(a['category_id'] for a in anns)) == 1)
print(f"\nImages with 1 category labeled: {single_cat/len(annotated)*100:.1f}%")

# 4. Object size -> informs imgsz and SAHI slice size
img_info = {img['id']: img for img in data['images']}
sizes_abs, sizes_rel = [], []
for ann in data['annotations']:
    img = img_info[ann['image_id']]
    bw, bh = ann['bbox'][2], ann['bbox'][3]
    sizes_abs.append(max(bw, bh))
    sizes_rel.append((bw * bh) / (img['width'] * img['height']))

print(f"\nMedian object side length (px): {np.median(sizes_abs):.0f}")
print(f"Objects with max side <64px: {sum(s<64 for s in sizes_abs)/len(sizes_abs)*100:.1f}%")
print(f"Median relative area: {np.median(sizes_rel)*100:.2f}%")
print(f"Objects <1% of image area: {sum(s<0.01 for s in sizes_rel)/len(sizes_rel)*100:.1f}%")

```

## Decision table from EDA

| EDA finding                 | Action                                                     |
|-----------------------------|------------------------------------------------------------|
| Classes with <100 examples  | RFS (Phase 3) + class-aware copy-paste (Phase 4) essential |
| >50% images single-category | Severe PU — prioritize Soft Teacher in Phase 3             |
| >30% objects <1% of image   | imgsz=1280 baseline, SAHI slice=640 in Phase 7             |
| Median object <64px         | Multi-scale training biased to scale=(0.3, 1.0)            |

## Stratified 5-fold split

```

from sklearn.model_selection import StratifiedKFold

# Stratify by rarest category present in each image
def rarest_class_in_image(img_id, anns_by_image, cat_counts):
    classes = set(a['category_id'] for a in anns_by_image[img_id])
    return min(classes, key=lambda c: cat_counts[c])

image_ids = list(img_info.keys())
strat_labels = [rarest_class_in_image(i, anns_by_image, cat_counts)
                 if anns_by_image.get(i) else -1
                 for i in image_ids]

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
folds = list(skf.split(image_ids, strat_labels))

# Save fold indices for Phase 6
import pickle
with open('data/cv_folds.pkl', 'wb') as f:
    pickle.dump(folds, f)

```

## Project structure

```

fathomnet-2026/
+--- data/
|   +--- rfs_repeat_factors.pkl
|   +--- cv_folds.pkl
|   +--- fathomnet_pseudo.yaml
+--- src/
|   +--- efl_loss.py           # Equalized Focal Loss
|   +--- pu_loss.py           # Kiryo PU loss
|   +--- hierarchical_loss.py
|   +--- soft_teacher.py      # EMA teacher-student
|   +--- rfs_sampler.py
|   +--- copy_paste.py        # Class-aware copy-paste
|   +--- multiscale_trainer.py
|   +--- underwater_preproc.py # CLAHE + gray world
|   +--- tta.py
|   +--- sahi_inference.py
|   +--- ensemble.py
|   +--- calibration.py
+--- configs/
|   +--- fathomnet.yaml
|   +--- taxonomy.py
+--- notebooks/
+--- submissions/

```

## Phase 1 — Self-Supervised Pretraining

**Days 3-5 • RunPod A100 80GB • ~15 hours • ~\$40 Data:** Full FathomNet DB (400k+ unlabeled marine images, downloadable via the FathomNet API)

### Why included despite BioClip2 shortcut

BioClip2 is a valid fallback, but full SSL pretraining on the complete FathomNet DB gives you: - A backbone genuinely tuned to your exact image distribution - The learning experience of setting up SimCLR/DINO properly - A publishable artifact if you decide to write the working note paper

### Dual backbone strategy — do both

Train **two** backbones so you can compare in Phase 2: 1. **SimCLR ResNet50** — battle-tested, fast, easy to integrate with YOLO 2. **DINOv2 ViT-S/14** — stronger representation, newer approach, more impressive for LLNL/MBARI conversations

Or skip DINOv2 and use BioClip2 as your “second backbone” to save ~5 hours.

### SimCLR implementation

```

# src/ssl_pretrain.py
import torch
import torch.nn as nn
import torchvision.models as models
from lightly.models.modules import SimCLRProjectionHead
from lightly.loss import NTXentLoss
from lightly.data import LightlyDataset
from lightly.transforms import SimCLRTransform

class UnderwaterSimCLR(nn.Module):
    def __init__(self):
        super().__init__()
        resnet = models.resnet50(pretrained=True)
        self.backbone = nn.Sequential(*list(resnet.children())[:-1])
        self.projection_head = SimCLRProjectionHead(2048, 2048, 128)

    def forward(self, x):
        return self.projection_head(self.backbone(x).flatten(1))

# Underwater-tuned augmentation
# Red absorbed first at depth -> strong blue/green shift
# Backscatter = noise; random depth = variable contrast
transform = SimCLRTransform(
    input_size=224,
    cj_prob=0.8, cj_strength=1.5,
    random_gray_scale=0.2, gaussian_blur=0.5,
    vf_prob=0.3, hf_prob=0.5, rr_prob=0.5,
)

dataset = LightlyDataset(input_dir="/workspace/fathomnet_full_db/", transform=transform)

# A100 80GB -> batch=512 (more neg pairs = stronger contrastive signal)
dataloader = torch.utils.data.DataLoader(
    dataset, batch_size=512, shuffle=True,
    num_workers=16, drop_last=True, pin_memory=True,
)

model = UnderwaterSimCLR().cuda()
criterion = NTXentLoss(temperature=0.07)
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)

wandb.init(project="fathomnet-2026", name="phase1-simclr-r50")
for epoch in range(100):
    total_loss = 0
    for (v1, v2), _, _ in dataloader:
        z1, z2 = model(v1.cuda()), model(v2.cuda())
        loss = criterion(z1, z2)
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        total_loss += loss.item()
    scheduler.step()
    wandb.log({"ssl_loss": total_loss/len(dataloader), "epoch": epoch})

torch.save(model.backbone.state_dict(),
            "/workspace/weights/simclr_fathomnet_r50.pt")

```

## DINOv2 fine-tuning (optional, ~5 hours)

```

# Fine-tune DINOv2 on FathomNet images using DINO objective
# Pretrained DINOv2 ViT-S/14 is already domain-general - we adapt it to marine
import torch
dinov2 = torch.hub.load('facebookresearch/dinov2', 'dinov2_vits14')
# Continue DINO training on FathomNet DB for ~50 epochs
# Save fine-tuned backbone to /workspace/weights/dinov2_fathomnet.pt

```

## BioClip2 alternative (no GPU, free)

```

from transformers import AutoModel
bioclip = AutoModel.from_pretrained("imageomics/bioclip-2")
# Save bioclip backbone weights
torch.save(bioclip.vision_model.state_dict(),
            "/workspace/weights/bioclip2_backbone.pt")

```

**Output:** Up to 3 backbone checkpoints in `/workspace/weights/`. Compared against ImageNet init in Phase 2.

## Phase 2 — Baselines

---

**Days 6-7 · Kaggle free T4 · ~4 GPU hours · \$0 Data:** 6,463 images, one val fold from the 5-fold split

### Exp 2.1 — Nano sanity check (~30 min)

```
from ultralytics import YOLO
import wandb

wandb.init(project="fathomnet-2026", name="exp2.1-nano-sanity")
YOLO("yolo11n.pt").train(
    data="fathomnet_fold0.yaml", epochs=10, imgsz=640,
    batch=32, device=0, name="exp2.1-nano-sanity"
)
```

### Exp 2.2 — Medium baseline (~2h) — REFERENCE POINT

```
wandb.init(project="fathomnet-2026", name="exp2.2-medium-baseline")
YOLO("yolo11m.pt").train(
    data="fathomnet_fold0.yaml",
    epochs=50, imgsz=640, batch=16, device=0,
    lr=0.01, lrf=0.01, momentum=0.937,
    weight_decay=0.0005, warmup_epochs=3,
    name="exp2.2-medium-baseline"
)
```

### Exp 2.3 — Backbone comparison bake-off (~1.5h)

Test ImageNet vs SimCLR vs DINOv2/BioClip2 backbones, same training config. Pick winner for all downstream experiments.

#### First submission

Submit best of 2.2/2.3. Record leaderboard mAP — your starting point.

**Output:** Baseline mAP, best backbone identified.

---

## Phase 3 — PU Learning + Class Imbalance

---

**Days 8-10 · RunPod RTX 4090 · ~12 GPU hours · ~\$9**

This is the densest phase. Three separate problems addressed here: PU labels, class imbalance, and modern semi-supervised detection.

### Exp 3.1 — Estimate PU prior $\pi$ (no GPU)

```
def estimate_pi(annotations, total_images, img_info, anns_by_image):
    labeled_ids = set(a['image_id'] for a in annotations)
    annotation_rate = len(labeled_ids) / total_images
    area_coverage = []
    for img_id in labeled_ids:
        img = img_info[img_id]
        ann_area = sum(a['bbox'][2] * a['bbox'][3] for a in anns_by_image[img_id])
        area_coverage.append(min(ann_area / (img['width'] * img['height']), 1.0))
    return annotation_rate * np.mean(area_coverage)

PI = estimate_pi(data['annotations'], len(data['images']), img_info, anns_by_image)
# Typical range 0.1-0.4
```

### Exp 3.2 — Kiryo PU loss (~2h)

Replaces default BCE for objectness with the unbiased PU estimator:

```
# src/pu_loss.py
class PUObjectnessLoss(nn.Module):
    def __init__(self, pi=0.2, beta=0.0, gamma=1.0):
        super().__init__()
        self.pi, self.beta, self.gamma = pi, beta, gamma

    def forward(self, pred_objectness, is_labeled):
        pos_loss = F.binary_cross_entropy_with_logits(
            pred_objectness[is_labeled],
            torch.ones_like(pred_objectness[is_labeled]))
        unlabeled_loss = F.binary_cross_entropy_with_logits(
            pred_objectness[~is_labeled],
            torch.zeros_like(pred_objectness[~is_labeled]))
        pu_loss = unlabeled_loss - self.pi * pos_loss
        if pu_loss.item() < -self.beta:
            pu_loss = -self.gamma * pu_loss
        return pos_loss + pu_loss
```

### Exp 3.3 — Equalized Focal Loss (EFL) (~2h) — NEW

EFL replaces standard focal loss with a **category-specific** focusing factor. Rare classes have more severe positive-negative imbalance than frequent ones, so they need stronger focusing.

```
# src/efl_loss.py
import torch
import torch.nn as nn
import torch.nn.functional as F

class EqualizedFocalLoss(nn.Module):
    """
    Equalized Focal Loss (Li et al. CVPR 2022).
    Per-class focusing factor  $\gamma_c$  scales with class rarity.
    """
    def __init__(self, num_classes=32, gamma_b=2.0, scale_factor=8.0,
                 class_counts=None):
        super().__init__()
        self.num_classes = num_classes
        self.gamma_b = gamma_b
        self.scale_factor = scale_factor

        # Compute per-class frequency statistic
        # Rarer classes get larger focusing factor  $\gamma$ 
        class_counts = torch.tensor(class_counts, dtype=torch.float32)
        self.register_buffer('class_counts', class_counts)
        # Gradient-based per-class  $\gamma$  (tracked during training)
        self.register_buffer('pos_grad', torch.zeros(num_classes))
        self.register_buffer('neg_grad', torch.zeros(num_classes))

    def forward(self, pred_logits, targets, reduction='mean'):
        """
        pred_logits: (N, C) classification logits
        targets: (N, C) one-hot or multi-label targets
        """
        # Per-class focusing factor:  $\gamma_c = \gamma_b + s * (1 - g_c)$ 
        # where  $g_c$  is the cumulative gradient ratio for class  $c$ 
        gradient_ratio = self.pos_grad / (self.neg_grad + 1e-8)
        gradient_ratio = gradient_ratio.clamp(0, 1)
        gamma_c = self.gamma_b + self.scale_factor * (1 - gradient_ratio)

        probs = torch.sigmoid(pred_logits)
        pt = probs * targets + (1 - probs) * (1 - targets)

        # Per-class focal weight
        gamma_expanded = gamma_c.unsqueeze(0) # (1, C)
        focal_weight = (1 - pt) ** gamma_expanded

        ce = F.binary_cross_entropy_with_logits(pred_logits, targets, reduction='none')
        loss = focal_weight * ce

        # Update gradient accumulators for next step
        with torch.no_grad():
            self.pos_grad += (targets * ce).sum(dim=0).detach()
            self.neg_grad += ((1-targets) * ce).sum(dim=0).detach()

        return loss.mean() if reduction == 'mean' else loss
```

Integrate via custom trainer that uses EFL for classification and PU loss for objectness.

### Exp 3.4 — Repeat Factor Sampling (~1h) — NEW

Oversample images containing rare classes. Pairs naturally with EFL.

```
# src/rfs_sampler.py
from torch.utils.data import WeightedRandomSampler
import pickle

def build_rfs_sampler(dataset, repeat_factors_path):
    with open(repeat_factors_path, 'rb') as f:
        image_rf = pickle.load(f)

    # Map dataset index -> image_id -> repeat factor
    weights = [image_rf.get(dataset.get_image_id(i), 1.0)
               for i in range(len(dataset))]

    return WeightedRandomSampler(weights, num_samples=len(dataset),
                                  replacement=True)

# In Ultralytics, override the default sampler in the data loader
```

### Exp 3.5 — Soft Teacher (~4h) — NEW, REPLACES PLAIN PSEUDO-LABELS

Soft Teacher is the modern version of pseudo-labeling. Instead of discrete rounds, an EMA-averaged “teacher” model generates soft pseudo-labels on-the-fly during training. The student learns from both labeled data AND the teacher’s confidence-weighted predictions every batch.

Key advantages over plain pseudo-labels: - Continuous adaptation, not discrete rounds - Confidence used as a soft weight, not a hard threshold - Teacher updates slowly, student updates fast — stable learning - 3-5 mAP improvement over plain pseudo-labeling on semi-sup COCO

```

# src/soft_teacher.py
import torch
from copy import deepcopy

class SoftTeacherTrainer:
    def __init__(self, student_model, ema_decay=0.9996,
                  conf_threshold=0.5, unlabeled_weight=4.0):
        self.student = student_model
        self.teacher = deepcopy(student_model)
        for p in self.teacher.parameters():
            p.requires_grad = False
        self.ema_decay = ema_decay
        self.conf_threshold = conf_threshold
        self.unlabeled_weight = unlabeled_weight

    def update_teacher(self):
        """EMA update: teacher ← decay * teacher + (1-decay) * student"""
        for t_param, s_param in zip(self.teacher.parameters(),
                                     self.student.parameters()):
            t_param.data = (self.ema_decay * t_param.data +
                            (1 - self.ema_decay) * s_param.data)

    def training_step(self, labeled_batch, unlabeled_batch):
        # Supervised loss on labeled data
        sup_loss = self.student.compute_loss(labeled_batch)

        # Generate soft pseudo-labels from teacher
        with torch.no_grad():
            self.teacher.eval()
            teacher_preds = self.teacher(unlabeled_batch['images'])

        # Filter high-confidence predictions, weight by confidence
        pseudo_boxes = []
        pseudo_weights = []
        for pred in teacher_preds:
            mask = pred['scores'] > self.conf_threshold
            pseudo_boxes.append({
                'boxes': pred['boxes'][mask],
                'labels': pred['labels'][mask],
            })
            # Soft weighting by confidence
            pseudo_weights.append(pred['scores'][mask])

        # Student learns from pseudo-labels with soft weights
        self.student.train()
        unsup_loss = self.student.compute_loss_with_weights(
            unlabeled_batch, pseudo_boxes, pseudo_weights)

        total_loss = sup_loss + self.unlabeled_weight * unsup_loss
        return total_loss

```

For the Ultralytics path, the practical approach is to use `mmdetection` or `detectron2`-based implementations (SoftTeacher repo has both) with YOLOv11's detection head swapped in, OR stick with Ultralytics and run 2 rounds of “poor man’s Soft Teacher” — iterative pseudo-labeling with EMA-averaged weights between rounds. The latter is more realistic given the timeline.

### Exp 3.6 — Combined: EFL + RFS + PU + Soft Teacher (~3h)

All four techniques applied together. This is the best-of-Phase-3 config that carries into Phase 4.

**Output:** Know which combination of {EFL, RFS, PU loss, Soft Teacher} gives best val mAP. Winner is baseline for all subsequent phases.

## Phase 4 — Architecture + Augmentation

Days 11-13 · RunPod RTX 4090 · ~20 GPU hours · ~\$15

### Exp 4.1 — Input resolution 1280 vs 640 (~2h)

### Exp 4.2 — Underwater preprocessing (~2h) — NEW

Apply before resizing, for both training and inference:

```

# src/underwater_preproc.py
import cv2
import numpy as np

def gray_world_correction(img):
    """Remove depth-dependent color cast via gray-world assumption."""
    img = img.astype(np.float32)
    channel_means = img.reshape(-1, 3).mean(axis=0)
    gray_mean = channel_means.mean()
    for c in range(3):
        img[:, :, c] *= gray_mean / (channel_means[c] + 1e-8)
    return np.clip(img, 0, 255).astype(np.uint8)

def clahe_enhance(img, clip_limit=2.0, tile_size=(8, 8)):
    """CLAHE on L channel to handle uneven ROV lighting."""
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_size)
    l_eq = clahe.apply(l)
    return cv2.cvtColor(cv2.merge([l_eq, a, b]), cv2.COLOR_LAB2BGR)

def underwater_preprocess(img):
    img = gray_world_correction(img)
    img = clahe_enhance(img)
    return img

# Apply via Ultralytics dataset hook or custom dataset class

```

### Exp 4.3 — Underwater augmentation (~2h)

```

YOLO("yolo11m.pt").train(
    data="fathomnet_pseudo.yaml",
    epochs=50, imgsz=640, batch=16, device=0,
    hsv_h=0.02, hsv_s=0.8, hsv_v=0.5,
    degrees=45, flipud=0.3, fliplr=0.5,
    mosaic=1.0, mixup=0.15,
    name="exp4.3-underwater-aug"
)

```

### Exp 4.4 — Class-aware copy-paste (~3h) — NEW

Standard copy-paste pastes random instances. **Class-aware copy-paste** biases toward rare classes:

```

# src/copy_paste.py
import numpy as np
import cv2
import random

class ClassAwareCopyPaste:
    """
    Copy-paste augmentation biased toward rare classes.
    Extracts instances from rare-class source images and pastes
    them into target images at random positions.
    """
    def __init__(self, annotations, images_dir, class_counts, prob=0.5,
                 rare_class_threshold=500):
        self.annotations = annotations
        self.images_dir = images_dir

        # Identify rare classes
        self.rare_classes = [c for c, n in class_counts.items()
                             if n < rare_class_threshold]

        # Build per-class instance bank (crops of rare-class objects)
        self.instance_bank = self._build_instance_bank()
        self.prob = prob

    def _build_instance_bank(self):
        bank = {c: [] for c in self.rare_classes}
        for ann in self.annotations:
            if ann['category_id'] in self.rare_classes:
                bank[ann['category_id']].append(ann)
        return bank

    def __call__(self, img, target_boxes, target_labels):
        if random.random() > self.prob:
            return img, target_boxes, target_labels

        # Pick a rare class weighted by inverse frequency
        rare_class = random.choice(self.rare_classes)
        if not self.instance_bank[rare_class]:
            return img, target_boxes, target_labels

        # Pick a random instance of that class
        src_ann = random.choice(self.instance_bank[rare_class])
        src_img = cv2.imread(f"{self.images_dir}/{src_ann['image_path']}")
        if src_img is None:
            return img, target_boxes, target_labels

        # Extract the instance crop
        x, y, w, h = [int(v) for v in src_ann['bbox']]
        instance = src_img[y:y+h, x:x+w]

        # Paste at random valid location (not overlapping existing annotations)
        H, W = img.shape[:2]
        for _ in range(10):
            px = random.randint(0, W - w)
            py = random.randint(0, H - h)
            new_box = [px, py, w, h]
            # Check non-overlap with existing targets
            if not any(iou_xywh(new_box, b) > 0.1 for b in target_boxes):
                img[py:py+h, px:px+w] = instance
                target_boxes.append(new_box)
                target_labels.append(rare_class)
                break

        return img, target_boxes, target_labels

```

Apply with `prob=0.5` -> half of training batches get a rare-class instance boosted in.

## Exp 4.5 — Multi-scale training (~3h) — NEW

Directly addresses the 2025 distribution shift (test objects smaller). Train at varying resolutions per batch:

```
# Ultralytics has multi-scale built in via `multi_scale=True`
YOLO("yolo11m.pt").train(
    data="fathomnet_pseudo.yaml",
    epochs=50, imgsz=1024, batch=8, device=0,
    multi_scale=True,      # randomly varies imgsz ±50% per batch
    scale=0.5,             # bias toward shrinking (0.5 vs default 0.5-1.5)
    name="exp4.5-multiscale"
)
```

If you want finer control, implement your own multi-scale trainer that samples from [640, 896, 1152, 1408] per batch.

## Exp 4.6 — Larger model YOLOv11l (~3h)

## Exp 4.7 — RT-DETR (~3h)

## Exp 4.8 — Multi-scale context crops (2025 winning insight) (~2h)

**Output:** Know best image size, best augmentation stack, best architecture. Top 2 configs carry to Phase 6.

# Phase 5 — Hierarchical Loss + Pseudo R2

Day 14 · Mix Kaggle T4 + RunPod · ~8 GPU hours · ~\$3

Marine taxonomy provides structure — confusing sea star <-> brittle star (both echinoderms) should cost less than sea star <-> jelly (different phyla).

## Taxonomy tree and hierarchical loss

```
# configs/taxonomy.py
FATHOMNET_TAXONOMY = {
    # Arthropods (32)
    0: 32, 2: 32, 10: 32, 13: 32, 26: 32, 29: 32,
    # Cnidarians (33)
    1: 33, 5: 33, 12: 33, 14: 33, 27: 33, 30: 33, 20: 33, 21: 33,
    # Echinoderms (34)
    7: 34, 11: 34, 19: 34, 25: 34, 31: 34,
    # Molluscs (35)
    4: 35, 16: 35, 22: 35, 23: 35,
    # Chordates (36)
    6: 36, 15: 36, 24: 36,
    # Annelids (37)
    3: 37,
    # Siphonophores (38)
    8: 38, 17: 38,
    # Other (39)
    18: 39, 28: 39,
}
```

```
# src/hierarchical_loss.py
class HierarchicalClassificationLoss(nn.Module):
    def __init__(self, taxonomy, num_classes=32, hier_weight=0.5):
        super().__init__()
        self.distance_matrix = self._build_distance_matrix(taxonomy, num_classes)
        self.hier_weight = hier_weight

    # ... (distance matrix building same as before)

    def forward(self, predictions, targets):
        ce_loss = F.cross_entropy(predictions, targets)
        probs = F.softmax(predictions, dim=-1)
        dist = self.distance_matrix.to(predictions.device)
        hier_loss = torch.stack([
            (probs[i] * dist[t]).sum() for i, t in enumerate(targets)
        ]).mean()
        return ce_loss + self.hier_weight * hier_loss
```

## Experiments

- **5.1** — Hierarchical loss only (~2h, Kaggle)
- **5.2** — EFL + hier loss combined (~2h, Kaggle)
- **5.3** — Soft Teacher round 2 using 5.2 model (~4h, RunPod)

**Output:** Final combined config. Feeds Phase 6.

## Phase 6 — 5-Fold Stratified Cross-Validation Training

**Days 15-18 · RunPod A100 40GB · ~40 GPU hours · ~\$80**

Train **5 YOLOv11m models on 5 different 80/20 splits**, plus **1 RT-DETR on all data** for architectural diversity. Six models total for the final ensemble.

Why not also YOLOv11l folds? Marginal gain on top of 5-fold YOLOv11m + RT-DETR (estimated +0.3-0.7 mAP for \$40 and 30 extra hours). The RT-DETR already provides the real architectural diversity (transformer vs CNN). Spend the money elsewhere or leave as buffer.

Benefits of 5-fold: - Each fold trains on 80% of data (5,170 images) -> near-optimal - More stable validation per experiment - 5 models with decorrelated errors -> free ensemble boost in Phase 7 - If one fold fails, you still have 4 others

### Training loop for folds

```
import pickle
with open('data/cv_folds.pkl', 'rb') as f:
    folds = pickle.load(f)

for fold_idx, (train_idx, val_idx) in enumerate(folds):
    write_fold_yaml(fold_idx, train_idx, val_idx)

wandb.init(project="fathomnet-2026",
            name=f"exp6-fold{fold_idx}-yolo11m-all-diff")
model = YOLO("yolo11m.pt")
model.load_pretrained_backbone("/workspace/weights/simclr_fathomnet_r50.pt")

model.train(
    data=f"fathomnet_fold{fold_idx}.yaml",
    epochs=150, imgsz=1280, batch=8, device=0,
    # All differentiators from Phases 3-5
    trainer=FullDifferentiatorTrainer, # EFL + PU + hier loss
    sampler=build_rfs_sampler('data/rfs_repeat_factors.pkl'),
    hsv_h=0.02, hsv_s=0.8, hsv_v=0.5,
    degrees=45, flipud=0.3, fliplr=0.5,
    mosaic=1.0, mixup=0.15, copy_paste=0.3,
    multi_scale=True, scale=0.5,
    cos_lr=True, close_mosaic=20,
    name=f"exp6-fold{fold_idx}"
)

shutil.copy(f"runs/exp6-fold{fold_idx}/weights/best.pt",
            f"/workspace/weights/fold{fold_idx}_best.pt")
```

### RT-DETR full-data run

```
YOLO("rtdetr-l.pt").train(
    data="fathomnet_full.yaml",
    epochs=150, imgsz=1280, batch=8, device=0,
    cos_lr=True,
    name="exp6-rtdetr-full"
)
```

### Budget breakdown for Phase 6

| Runs                 | Time per run | Subtotal                       |
|----------------------|--------------|--------------------------------|
| 5 × YOLOv11m folds   | ~6h each     | 30h                            |
| 1 × RT-DETR all-data | ~8h          | 8h                             |
| <b>Total Phase 6</b> |              | <b>~38h × \$1.99/hr ≈ \$76</b> |

**Output: 6 trained models** in /workspace/weights/ : - fold0\_best.pt through fold4\_best.pt (5 YOLOv11m folds) - rtdetr\_full\_best.pt (1 RT-DETR)

This 6-model ensemble powers Phase 7.

**If Phase 6 completes with spare budget,** the single best use of the extra money is NOT more models — it’s per-class threshold tuning (backup plan item #1) which needs no GPU and often adds +0.5-1 mAP. Go there instead of training more models.

## Phase 7 — SAHI + TTA + Ensemble + Calibration

Day 19 · RunPod RTX 4090 · ~15 GPU hours · ~\$11

Four inference-time techniques stacked, each adds mAP for free (no retraining).

### Differentiator: SAHI tiled inference — NEW

Directly addresses the 2025 distribution shift. Slice each test image into overlapping 640×640 tiles, run detector on each, merge with NMS.

```
# src/sahi_inference.py
from sahi import AutoDetectionModel
from sahi.predict import get_sliced_prediction
from sahi.utils.cv import read_image

def sahi_predict(model_path, image_path,
                 slice_height=640, slice_width=640,
                 overlap=0.2, conf=0.1):
    detection_model = AutoDetectionModel.from_pretrained(
        model_type='yolov8', # sahi treats yolov11 as yolov8 interface
        model_path=model_path,
        confidence_threshold=conf,
        device="cuda",
    )

    result = get_sliced_prediction(
        image=image_path,
        detection_model=detection_model,
        slice_height=slice_height,
        slice_width=slice_width,
        overlap_height_ratio=overlap,
        overlap_width_ratio=overlap,
        postprocess_type="GREEDYNMM",
        postprocess_match_threshold=0.5,
    )

    return result.object_prediction_list

# Use slice size matching typical object size
# If median object ~80px in a 1920px image -> slice ~640px gives objects ~25% of tile
```

SAHI alone typically adds 2-5 mAP when objects are small. For FathomNet given the 2025 note, expect at least +2 mAP.

### Differentiator: Test-Time Augmentation

```
# src/tta.py
from ensemble_boxes import weighted_boxes_fusion

def predict_tta(model, image_path, conf=0.01):
    img = cv2.imread(image_path); h, w = img.shape[:2]

    augs = [
        ('original', lambda x: x),
        ('hflip', lambda x: cv2.flip(x, 1)),
        ('vflip', lambda x: cv2.flip(x, 0)),
        ('rot90cw', lambda x: cv2.rotate(x, cv2.ROTATE_90_CLOCKWISE)),
        ('rot90ccw', lambda x: cv2.rotate(x, cv2.ROTATE_90_COUNTERCLOCKWISE)),
    ]

    # Apply inverse box transforms to bring predictions back to original space
    # ... (as in v1)

    return fused_boxes, fused_scores, fused_labels
```

### Differentiator: 6-model WBF ensemble

```

# src/ensemble.py
def ensemble_all_models(test_images_dir):
    # 5 YOLOv11m folds + 1 RT-DETR
    model_paths = [
        f"/workspace/weights/fold{i}_best.pt" for i in range(5)
    ] + ["/workspace/weights/rt detr_full_best.pt"]

    # Weight folds equally; weight RT-DETR based on solo leaderboard score
    weights = [1.0]*5 + [1.2] # tune by experiment

    for img_path in Path(test_images_dir).glob("*.jpg"):
        # For each model, apply SAHI + TTA
        boxes_per_model = []
        for m_path in model_paths:
            sahi_preds = sahi_predict(m_path, img_path)
            tta_preds = predict_tta(YOLO(m_path), img_path)
            # Merge SAHI and TTA within model
            merged = weighted_boxes_fusion(...)
            boxes_per_model.append(merged)

        # Final WBF across all 6 models
        final = weighted_boxes_fusion(
            boxes_per_model, weights=weights,
            iou_thr=0.5, skip_box_thr=0.05
        )

```

## Differentiator: Isotonic score calibration

```

from sklearn.isotonic import IsotonicRegression

def calibrate_scores(val_preds, val_gt, iou_threshold=0.5):
    raw, correct = [], []
    for image_id, preds in val_preds.items():
        gt_boxes = val_gt.get(image_id, [])
        for pred in preds:
            raw.append(pred['score'])
            best_iou = max(
                (compute_iou(pred['bbox'], gt['bbox']) for gt in gt_boxes
                 if gt['category_id'] == pred['category_id']),
                default=0.0)
            correct.append(1 if best_iou >= iou_threshold else 0)
    cal = IsotonicRegression(out_of_bounds='clip')
    cal.fit(raw, correct)
    return cal

# Fit on aggregated fold-out-of-fold predictions (free since you did 5-fold CV)
# Apply to test predictions

```

## Experiments in Phase 7

- **7.1** — Best single model + TTA only (~1h) — isolates TTA contribution
- **7.2** — Best single model + SAHI only (~1h) — isolates SAHI contribution
- **7.3** — Best single model + SAHI + TTA (~1h) — combined inference
- **7.4** — 5-fold YOLO ensemble + SAHI + TTA (~4h)
- **7.5** — 6-model (5 YOLO + RT-DETR) ensemble + SAHI + TTA + calibration (~6h) — **FINAL**
- **7.6** — Buffer: tune ensemble weights, per-class confidence thresholds, NMS IoU (~2h)

Submit each. Track which improvements actually help on leaderboard.

**Output:** final\_submission.csv

## Full Experiment Tracker

| Exp | Phase | Model                   | Data  | PU    | CLS Imb | Aug     | Hier | Val mAP     | LB mAP      |
|-----|-------|-------------------------|-------|-------|---------|---------|------|-------------|-------------|
| 2.1 | 2     | YOLOv11n                | fold0 | [NO]  | [NO]    | default | [NO] | -           | -           |
| 2.2 | 2     | YOLOv11m                | fold0 | [NO]  | [NO]    | default | [NO] | <b>BASE</b> | <b>BASE</b> |
| 2.3 | 2     | YOLOv11m<br>+ backbones | fold0 | [NO]  | [NO]    | default | [NO] | ?           | ?           |
| 3.2 | 3     | + Kiryo PU              | fold0 | Kiryo | [NO]    | default | [NO] | ?           | -           |
| 3.3 | 3     | + FFI                   | fold0 | Kiryo | FFI     | default | [NO] | ?           | -           |

| id       | year | feature                     | loss   | kernel   | EFL     | dataset  | [RFS] | ?          |          |
|----------|------|-----------------------------|--------|----------|---------|----------|-------|------------|----------|
| 3.4      | 3    | + RFS sampler               | fold0  | Kiryo    | EFL+RFS | default  | [NO]  | ?          | -        |
| 3.5      | 3    | + Soft Teacher              | +unlab | Kiryo+ST | EFL+RFS | default  | [NO]  | ?          | ?        |
| 3.6      | 3    | All combined                | +unlab | Kiryo+ST | EFL+RFS | default  | [NO]  | ?          | ?        |
| 4.1      | 4    | imgsz=1280                  | +unlab | best     | best    | default  | [NO]  | ?          | -        |
| 4.2      | 4    | UW preproc                  | +unlab | best     | best    | +preproc | [NO]  | ?          | -        |
| 4.3      | 4    | UW aug                      | +unlab | best     | best    | UW       | [NO]  | ?          | -        |
| 4.4      | 4    | +class copy-paste           | +unlab | best     | best+CP | UW       | [NO]  | ?          | -        |
| 4.5      | 4    | +multi-scale                | +unlab | best     | best+CP | UW+MS    | [NO]  | ?          | -        |
| 4.6      | 4    | YOLOv11l                    | +unlab | best     | best+CP | UW+MS    | [NO]  | ?          | -        |
| 4.7      | 4    | RT-DETR-L                   | +unlab | best     | best+CP | UW+MS    | [NO]  | ?          | -        |
| 4.8      | 4    | context crops               | +unlab | best     | best+CP | UW+MS    | [NO]  | ?          | -        |
| 5.1      | 5    | +hier loss                  | +unlab | best     | best+CP | best     | [YES] | ?          | -        |
| 5.2      | 5    | hier+EFL+PU                 | +unlab | best     | best+CP | best     | [YES] | ?          | ?        |
| 5.3      | 5    | + Soft T R2                 | +STR2  | best     | best+CP | best     | [YES] | ?          | ?        |
| 6.0-6.4  | 6    | YOLOv11m folds 0-4          | 5-fold | all      | all     | all      | [YES] | ? per fold | -        |
| 6.rtdetr | 6    | RT-DETR-L all data          | all    | all      | all     | all      | [YES] | -          | ?        |
| 7.1      | 7    | best + TTA                  | test   | -        | -       | -        | -     | -          | ?        |
| 7.2      | 7    | best + SAHI                 | test   | -        | -       | -        | -     | -          | ?        |
| 7.3      | 7    | best + SAHI + TTA           | test   | -        | -       | -        | -     | -          | ?        |
| 7.4      | 7    | 5-fold + SAHI + TTA         | test   | -        | -       | -        | -     | -          | ?        |
| 7.5      | 7    | <b>FINAL: 6-model + all</b> | test   | -        | -       | -        | -     | -          | <b>?</b> |

## Complete Differentiator Inventory

| # | Differentiator                                       | Phase | Impact                     | New?       |
|---|------------------------------------------------------|-------|----------------------------|------------|
| 1 | <b>Kiryo PU loss</b>                                 | 3     | Core competition challenge |            |
| 2 | <b>Equalized Focal Loss (EFL)</b>                    | 3     | Class imbalance            | <b>NEW</b> |
| 3 | <b>Repeat Factor Sampling (RFS)</b>                  | 3     | Rare class oversampling    | <b>NEW</b> |
| 4 | <b>Soft Teacher (vs plain pseudo)</b>                | 3+5   | Modern semi-sup detection  | <b>NEW</b> |
| 5 | <b>Hierarchical taxonomy loss</b>                    | 5     | 2025 winning insight       |            |
| 6 | <b>Underwater preprocessing (CLAHE + gray-world)</b> | 4+7   | Domain-specific            | <b>NEW</b> |
| 7 | <b>Class-aware copy-paste</b>                        | 4     | Rare class augmentation    | <b>NEW</b> |
| 8 | <b>Multi-scale training</b>                          | 4     | 2025 distribution shift    | <b>NEW</b> |
|   |                                                      |       |                            |            |

|    |                                                 |     |                                    |            |
|----|-------------------------------------------------|-----|------------------------------------|------------|
| 9  | <b>Context crops (multi-scale inputs)</b>       | 4   | 2025 winning insight               |            |
| 10 | <b>SSL pretraining (SimCLR/DINOv2/BioClip2)</b> | 1   | Domain-pretrained backbone         |            |
| 11 | <b>5-fold stratified CV + ensemble</b>          | 6+7 | Better validation + free ensemble  | <b>NEW</b> |
| 12 | <b>RT-DETR in ensemble</b>                      | 6+7 | Architectural diversity            |            |
| 13 | <b>SAHI tiled inference</b>                     | 7   | Small objects + distribution shift | <b>NEW</b> |
| 14 | <b>TTA (5 augmentations + WBF)</b>              | 7   | Free mAP                           |            |
| 15 | <b>6-model WBF ensemble</b>                     | 7   | Architectural + fold diversity     |            |
| 16 | <b>Isotonic confidence calibration</b>          | 7   | mAP score ordering                 |            |

16 distinct differentiators. Most competitors will have 2-3 of these.

## Rules

1. **One variable per experiment.** Otherwise no attribution.
2. **Save weights to** `/workspace/weights/` **after every promising run.** Kaggle resets, RunPod pods delete.
3. **Study failure cases after each phase.** Which categories have lowest AP? Where does the model miss? Where does it hallucinate? Patterns tell you what to try next.
4. **Read the 2025 winning solution.** Posted on the FathomNet 2025 Kaggle discussion board.
5. **Stop RunPod pods when idle.** Running = charging even at rest.
6. **Submit early and often.** Every submission validates the pipeline.
7. **The backup plan is optional polish.** Don't touch it until the master plan is complete.

## Next Immediate Actions (Today, April 17)

1. Register for CLEF ([clef-labs-registration.dipintra.it](https://clef-labs-registration.dipintra.it))
2. Join the Kaggle competition
3. RunPod account with \$100 credit, SSH key added
4. W&B account, API key noted
5. Cursor + Remote-SSH extension installed
6. Download FathomNet 2026 dataset via Kaggle API
7. Start Phase 0 EDA — run the exploration script

Proceed to Phase 1 only when Phase 0 deliverables exist: - `data/rfs_repeat_factors.pkl` - `data/cv_folds.pkl` - `configs/fathomnet.yaml` - `configs/taxonomy.py` - Clear numbers on PU severity and object size distribution

## References

- **Kiryo et al. 2017** — “Positive-Unlabeled Learning with Non-Negative Risk Estimator” (NeurIPS). PU loss theory.
- **Li et al. 2022** — “Equalized Focal Loss for Dense Long-Tailed Object Detection” (CVPR). EFL paper.
- **Xu et al. 2021** — “End-to-End Semi-Supervised Object Detection with Soft Teacher” (ICCV). Soft Teacher paper.
- **Gupta et al. 2019** — “LVIS: A Dataset for Large Vocabulary Instance Segmentation”. RFS origin.
- **Akyon et al. 2022** — “Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection”. SAHI.
- **Caron et al. 2021** — “Emerging Properties in Self-Supervised Vision Transformers (DINO)”.
- **FathomNet 2025 winning solution** — Kaggle discussion board.
- **Ultralytics docs** — [docs.ultralytics.com](https://docs.ultralytics.com)
- **BioClip2** — [imageomics/bioclclip-2](https://imageomics/bioclclip-2) on Hugging Face